Optimization - I

Dr. Rajdip Nayek

Assistant Professor

Department of Applied Mechanics

Indian Institute of Technology Delhi

E-mail: rajdipn@iitd.ac.in

Key components of any ML algorithm

1. The ***data*** that we learn from

input features

output labels

e.g. $(\underset{\sim}{x}^{(i)}, t^{(i)})$ in supervised learning

2. A ***model*** with parameters to predict the output

e.g. $y = h_{\underset{\sim}{\theta}}(\underset{\sim}{x})$

3. An ***loss*** **function** that quantifies how bad the model is doing

e.g. $\mathcal{L} = \frac{1}{N_{train}} \sum_{i=1} (t^{(i)} - y^{(i)})^2$

4. An ***optimization*** **algorithm** to adjust the model's parameters to minimize the loss function

e.g. $\underset{\sim}{\theta}^{new} \leftarrow \underset{\sim}{\theta}^{old} - \eta \frac{\partial \mathcal{L}}{\partial \underset{\sim}{\theta}}$

Overview

- **Training examples:** $\{(\boldsymbol{x}^{(1)}, t^{(1)}), (\boldsymbol{x}^{(2)}, t^{(2)}), \cdots, (\boldsymbol{x}^{(N)}, t^{(N)})\}$
- **Model:** $y = h_{\boldsymbol{\theta}}(\boldsymbol{x})$, we accumulate all weights and biases into a vector $\boldsymbol{\theta}$

- **Loss function** → Average over individual training losses

$$\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N} \ell^{(i)} = \frac{1}{N}\sum_{i=1}^{N} \ell\left(y^{(i)}, t^{(i)}\right)$$

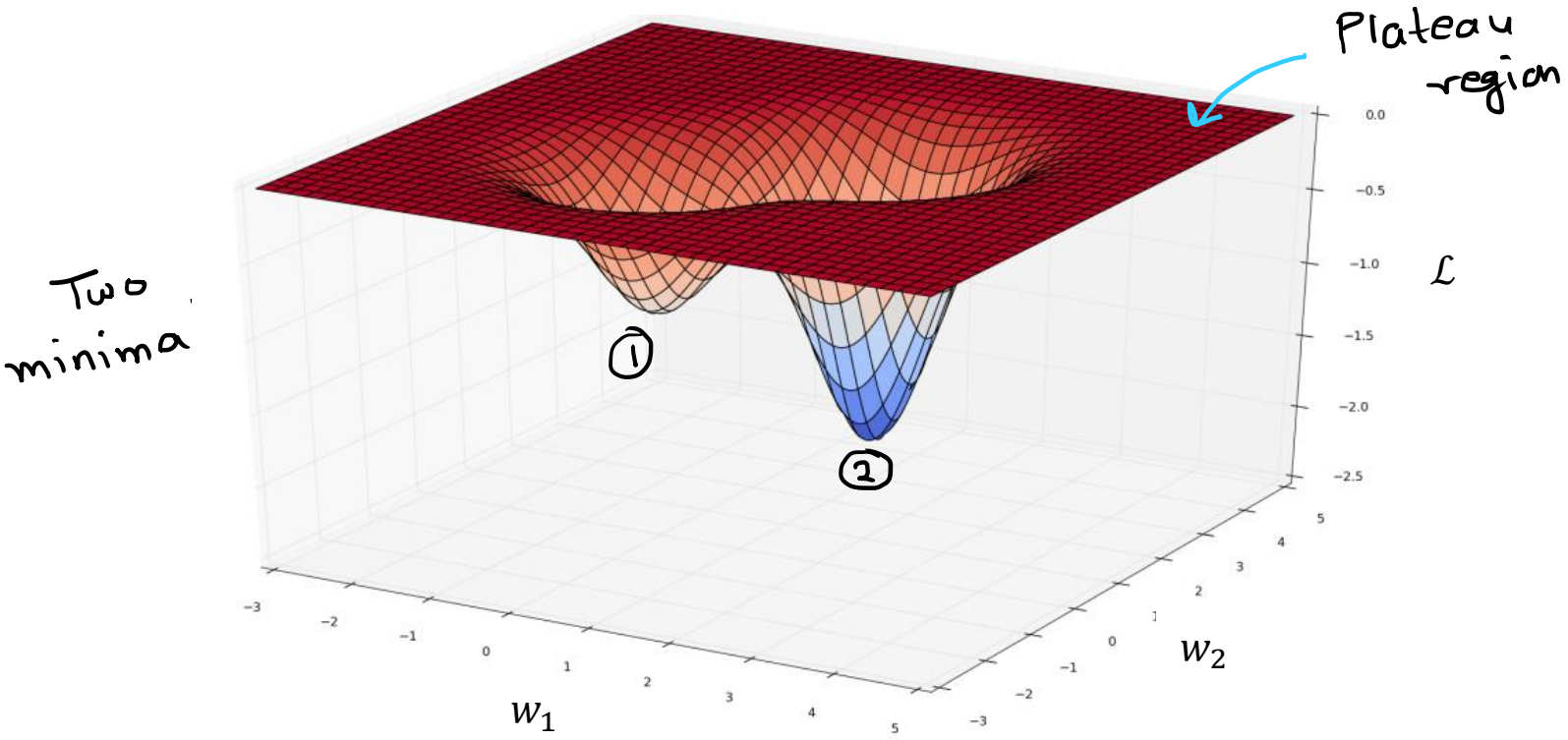
- **Gradient of loss function** w.r.t parameter $\boldsymbol{\theta}$

$$\nabla_{\boldsymbol{\theta}}\mathcal{L} = \nabla_{\boldsymbol{\theta}} \frac{1}{N}\sum_{i=1}^{N} \ell^{(i)}$$
$$= \frac{1}{N}\sum_{i=1}^{N} \nabla_{\boldsymbol{\theta}}\ell^{(i)}$$

- In the last lecture, we talked about how to compute gradients $\nabla_{\boldsymbol{\theta}}\ell^{(i)}$ of the individual loss functions w.r.t. parameter vector $\boldsymbol{\theta}$ using **backprop**
- In this lecture, we will talk about optimization of neural nets

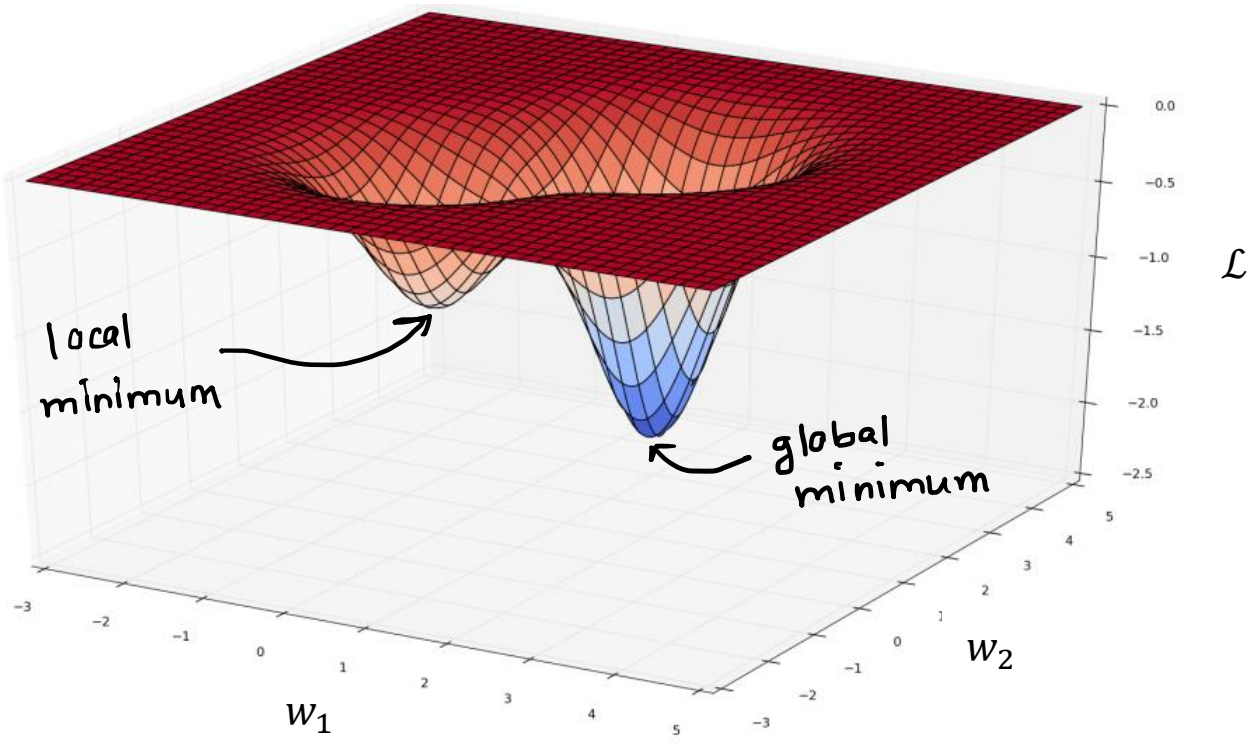
Basics: Visualizing a loss surface

- When we train a neural network, we're trying to minimize some loss function $\mathcal{L}$, which is a function of the network's parameters, which we'll denote with the vector $\boldsymbol{\theta}$
- $\boldsymbol{\theta}$ contains all the neural network weights and biases
- Suppose $\boldsymbol{\theta}$ consists of two weights, w_1 and w_2. Let's visualize a loss surface $\mathcal{L}$



Basics: Visualizing a loss surface

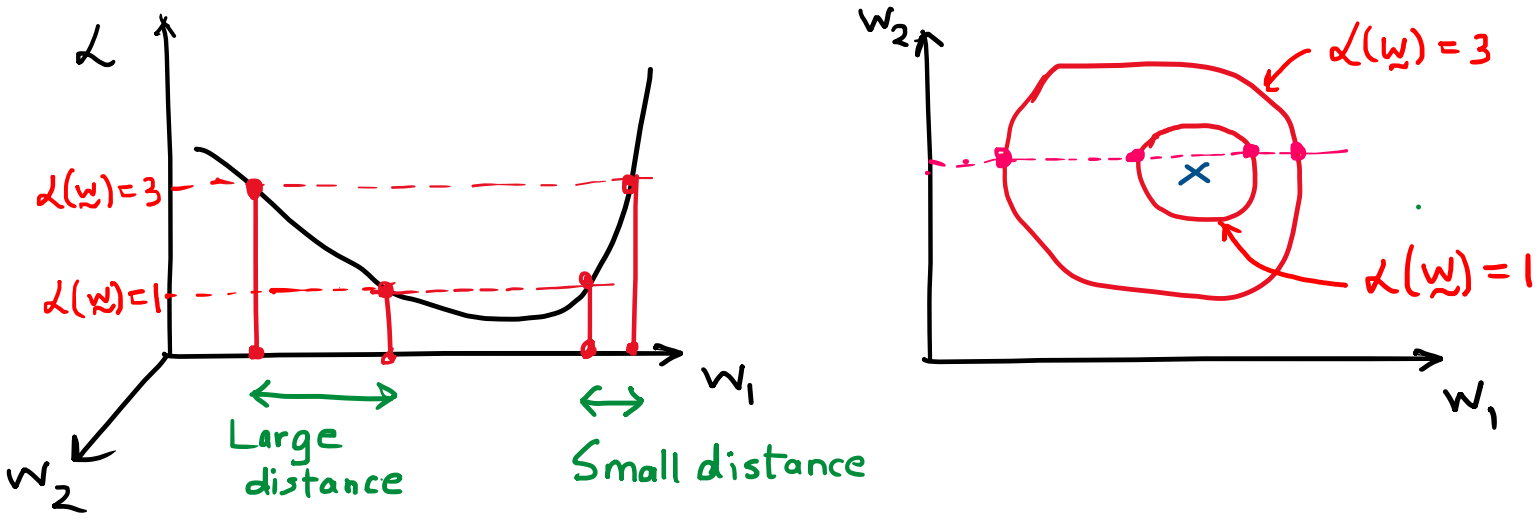
- **Minima** are points that minimize the loss function within a small neighbourhood
- One of them is a **local minimum**
- Another one is a **global minimum**, a point which achieves the minimum loss over all values of $\boldsymbol{\theta}$



Basics: Contour plots

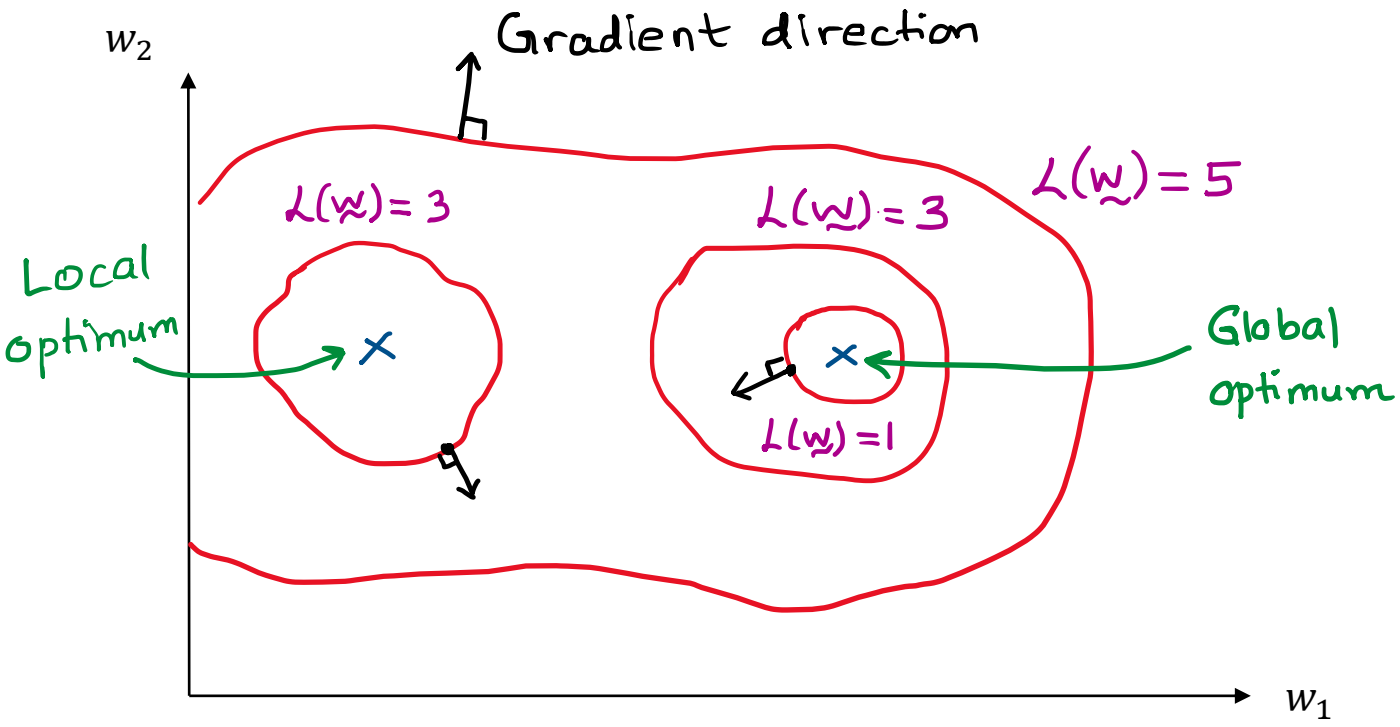
- *Visualizing a surface in 3D can sometimes become a bit cumbersome*
- *Can we do a 2D visualization of the loss surface?*
- *Yes, let's take a look at something known as* ***contours***

Basics: Contour plots



- Each of the contours is associated with a set of parameters where the loss function takes a particular value
- A **small distance** between the contours indicates a **steep slope** along that direction
- A **large distance** between the contours indicates a **gentle slope** along that direction

Basics: Contour plots



- Can't determine the magnitude of the gradient from the contour plot
- Can determine the **gradient direction**: *the gradient is always orthogonal (perpendicular) to the contours*
- Recall: Gradient descent would happen in opposite direction of gradient

Overview

- **Training examples:** $\{(\boldsymbol{x}^{(1)}, t^{(1)}), (\boldsymbol{x}^{(2)}, t^{(2)}), \cdots, (\boldsymbol{x}^{(N)}, t^{(N)})\}$
- **Model:** $y = h_{\boldsymbol{\theta}}(\boldsymbol{x})$, we accumulate all weights and biases into a vector $\boldsymbol{\theta}$

- **Loss function** → Average over individual training losses

$$\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N} \ell^{(i)} = \frac{1}{N}\sum_{i=1}^{N} \ell\left(y^{(i)}, t^{(i)}\right)$$

- **Gradient of loss function** w.r.t parameter $\boldsymbol{\theta}$

$$\nabla_{\boldsymbol{\theta}}\mathcal{L} = \nabla_{\boldsymbol{\theta}} \frac{1}{N}\sum_{i=1}^{N} \ell^{(i)}$$
$$= \frac{1}{N}\sum_{i=1}^{N} \nabla_{\boldsymbol{\theta}}\ell^{(i)}$$

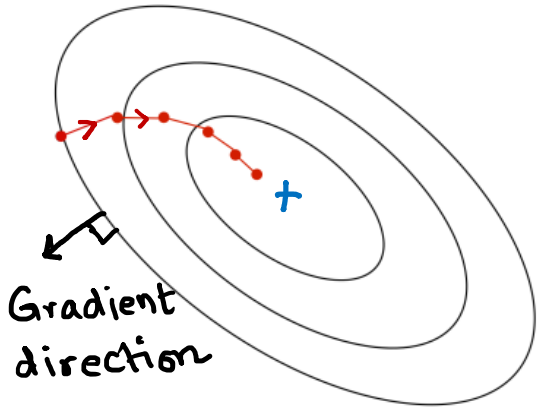
- In the last lecture, we talked about how to compute gradients $\nabla_{\boldsymbol{\theta}}\ell^{(i)}$ of the individual loss functions w.r.t. parameter vector $\boldsymbol{\theta}$ using **backprop**
- In this lecture, we will talk about optimization of neural nets

Batch Gradient Descent

$$\nabla_{\boldsymbol{\theta}}\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N}\nabla_{\boldsymbol{\theta}}\ell^{(i)} \qquad \boldsymbol{\theta}^{new} \leftarrow \boldsymbol{\theta}^{old} - \eta\ \nabla_{\boldsymbol{\theta}}\mathcal{L}\Big|_{\boldsymbol{\theta}=\boldsymbol{\theta}^{old}}$$

- Compute the total gradient $\nabla_{\boldsymbol{\theta}}\mathcal{L}$ by averaging over *all* individual gradients for every training example, and then update the parameters
- The algorithm goes over the entire data once before updating the parameters

- This is known as **batch gradient descent (BGD)**, since we treat the entire training set as a batch
- **Pros**: There is no approximation. Each update step guarantees that the loss will decrease
- **Cons**: However, BGD can be very time-consuming for a large dataset



Batch Gradient Descent

$$\nabla_{\boldsymbol{\theta}}\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N}\nabla_{\boldsymbol{\theta}}\ell^{(i)}$$

$$\boldsymbol{\theta}^{new} \leftarrow \boldsymbol{\theta}^{old} - \eta\ \nabla_{\boldsymbol{\theta}}\mathcal{L}\Big|_{\boldsymbol{\theta}=\boldsymbol{\theta}^{old}}$$

- Batch gradient descent treat the entire training set as a single batch
- Updates the parameter vector after each full pass (epoch) over the entire dataset


```
theta   = -1        # initialize parameter vector
eta     = 0.001     # learning rate
epochs  = 100       # number of passes over entire dataset
Ntr     = 10000     # number of training points
for i in range(epochs):
  dtheta = 0        # initialize increment to zero
  for x,t in zip(X,T):
    dtheta += grad_theta(theta, x, t)

  theta = theta - eta * dtheta / Ntr
```

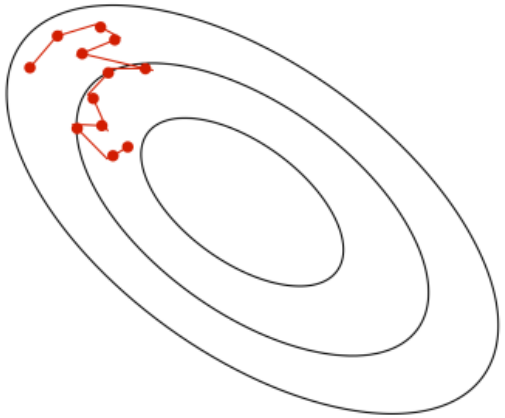
Stochastic Gradient Descent

- **Stochastic gradient descent**: we can use a noisy (or stochastic) estimate of the gradient from a single training example to update the parameter vector $\boldsymbol{\theta}$

$$\nabla_{\boldsymbol{\theta}}\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N} \nabla_{\boldsymbol{\theta}}\ell^{(i)} \qquad \boldsymbol{\theta}^{new} \leftarrow \boldsymbol{\theta}^{old} - \eta \left.\nabla_{\boldsymbol{\theta}}\ell^{(i)}\right|_{\boldsymbol{\theta}=\boldsymbol{\theta}^{old}}$$

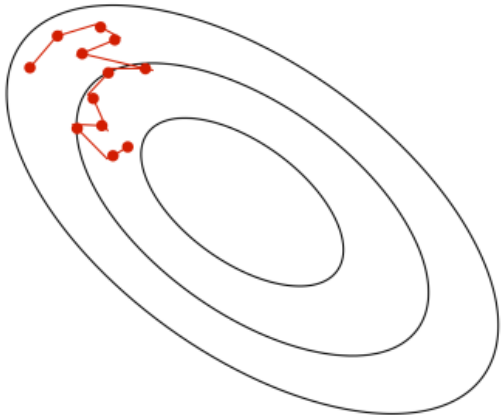
- The algorithm updates the parameters for every single data point
- **Pros**: SGD can make significant progress before it has even looked at all the data!
- **Cons:** It is an approximate (rather stochastic) gradient. No guarantee that each step will decrease the loss

```
theta, eta, epochs  = -1, 0.001, 100
for i in range(epochs):
  dtheta = 0        # initialize increment to zero
  for x,t in zip(X,T):
    dtheta = grad_theta(theta, x, t)
    theta  = theta - eta * dtheta
```



Stochastic Gradient Descent

- We see many **fluctuations**. Why ? Because we are making greedy decisions
- Each data point is trying to push the parameters in a direction most favorable to it (without being aware of how the parameter update affects other points)
- A parameter update which is locally favorable to one point may harm other points (its almost as if the data points are competing with each other)
- There is no guarantee that each local greedy move will reduce the global error
- Can we reduce the oscillations by improving our stochastic estimates of the gradient (currently estimated from just 1 data point at a time)?
- Yes, let's look at mini-batch SGD



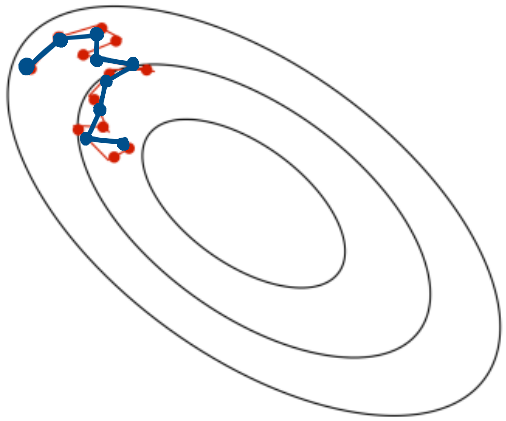
Mini-batch Gradient Descent

- Compute the gradients on a medium-sized set of training examples, called a *mini-batch*
- Note that the algorithm updates the parameters after it sees a batch size B number of data points
- The stochastic estimates of gradients here are slightly better and less noisy

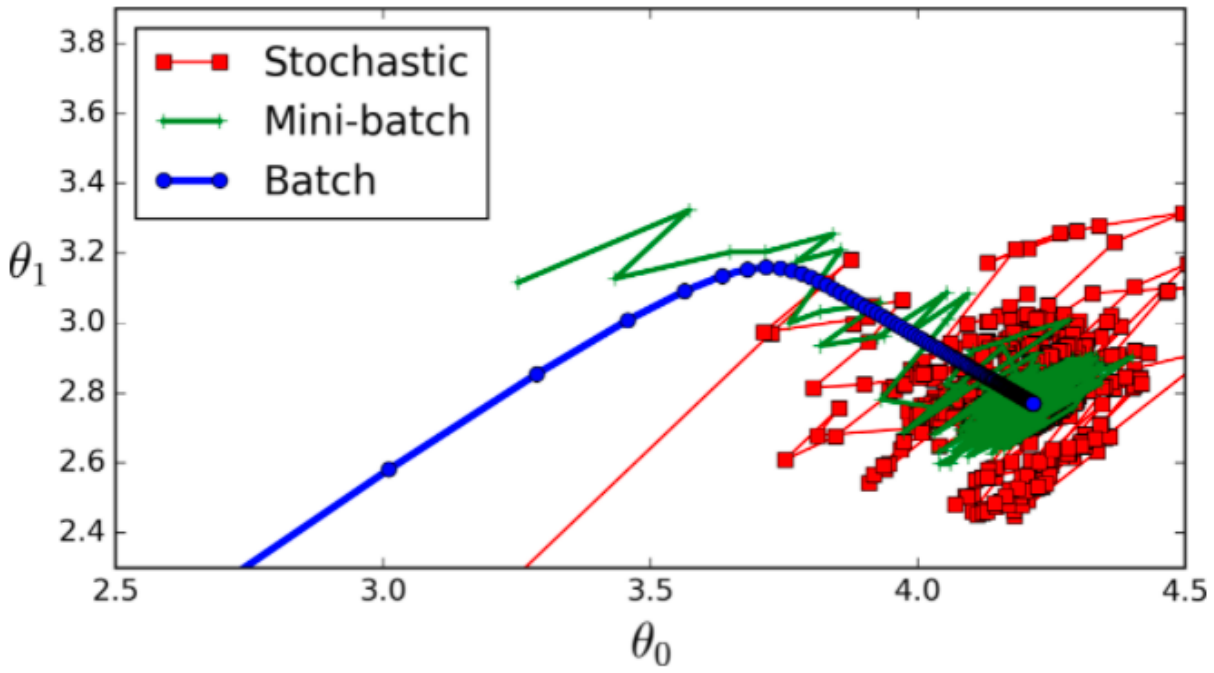
```
theta, eta, epochs  = -1, 0.001, 100
batch_size          = 64
num_points_seen     = 0
for i in range(epochs):
  dtheta = 0
  for x,t in zip(X,T):
    dtheta += grad_theta(theta, x, t)
    num_points_seen += 1

    if num_points_seen % batch_size == 0:
      # seen one mini-batch
      theta = theta - eta * dtheta / batch_size
      dtheta = 0 # reset gradients
```

- Typical batch sizes are 64, 128, 256



Mini-batch Gradient Descent performance



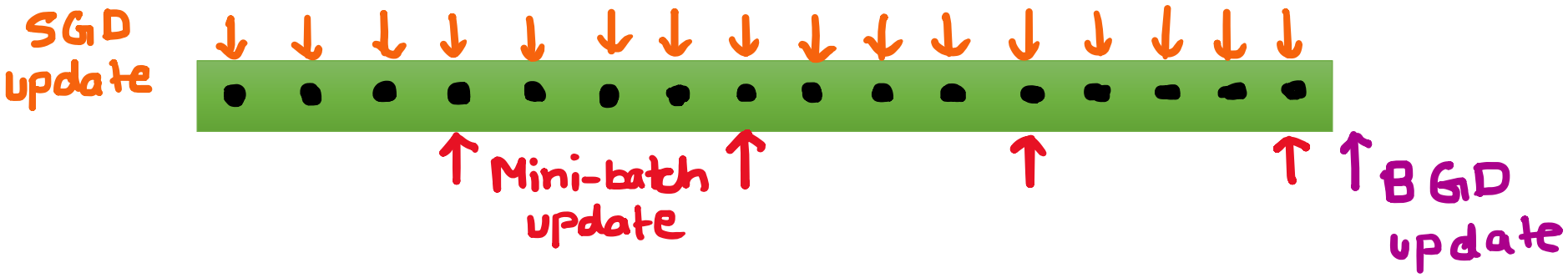
The mini-batch size *B* is a hyperparameter that needs to be set

- **Large batches:** converge in fewer parameter updates because each stochastic gradient is less noisy

- **Small batches:** perform more parameter updates because each one requires less computation

Things to remember

- N is the total number of training examples
- B is the mini batch size
- 1 epoch = one pass over the entire data
- 1 iteration = one update step of the parameters



Algorithm	Batch size	Number of iterations in 1 epoch
Batch GD	N	1
SGD	1	N
Mini-batch GD	B	N/B

PyTorch code mini-batch SGD

```
import torch
from torch.utils.data import DataLoader, TensorDataset

# Create the dataset with N trainig samples
# Training samples, input dim, hidden dim, output dim
N, D_in, H, D_out = 10000, 1000, 100, 10

x = torch.randn(N, D_in)
y = torch.randn(N, D_out)

# Define the batch size and the number of epochs
BATCH_SIZE = 64
EPOCHS = 100

# Use torch.utils.data to create a DataLoader
# that will take care of creating batches
dataset = TensorDataset(x, y)
dataloader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)

# Define model, loss and optimizer
model = torch.nn.Sequential(
    torch.nn.Linear(D_in, H),
    torch.nn.ReLU(),
    torch.nn.Linear(H, D_out),
)
```

PyTorch code mini-batch SGD

```
loss_fn   = torch.nn.MSELoss()
optimizer = torch.optim.SGD(model.parameters(), lr=1e-3)

# Get the dataset size for printing (it is equal to N)
dataset_size = len(dataloader.dataset)

# Loop over epochs
for epoch in range(EPOCHS):

    # Loop over batches in an epoch using DataLoader
    for id_batch, (x_batch, t_batch) in enumerate(dataloader):

        y_batch = model(x_batch)  # input x and predict based on x

        loss = loss_fn(y_batch, t_batch)

        optimizer.zero_grad()     # clear gradients for next train
        loss.backward()           # backpropagation
        optimizer.step()          # update parameters
```

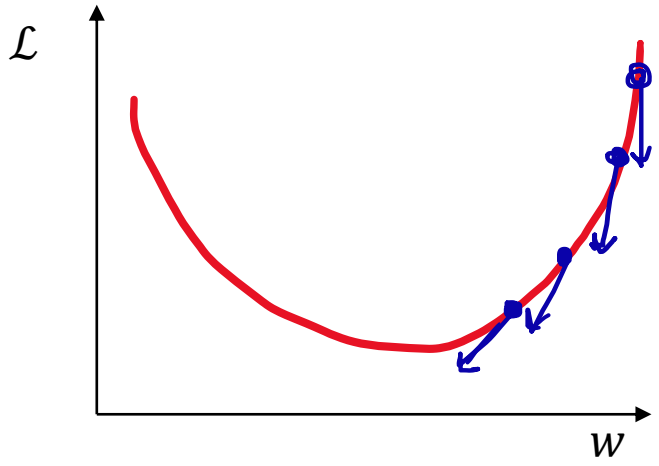
Gradient descent-based optimizers

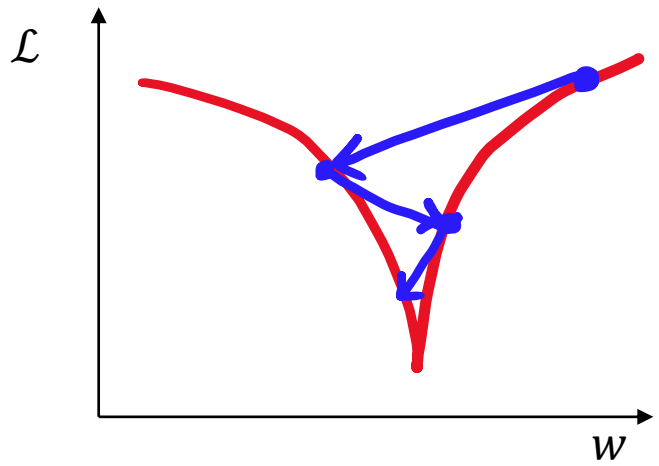
Problems and workarounds

Local Optima

Local optima

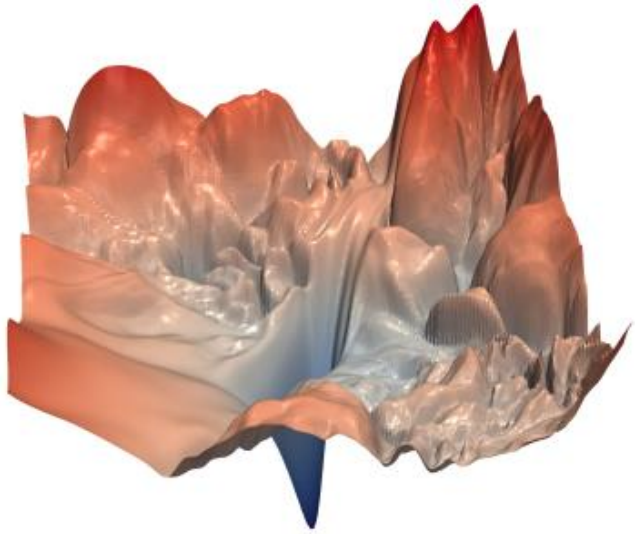
- If a function is convex, it has **a global minimum** and **no local minima**





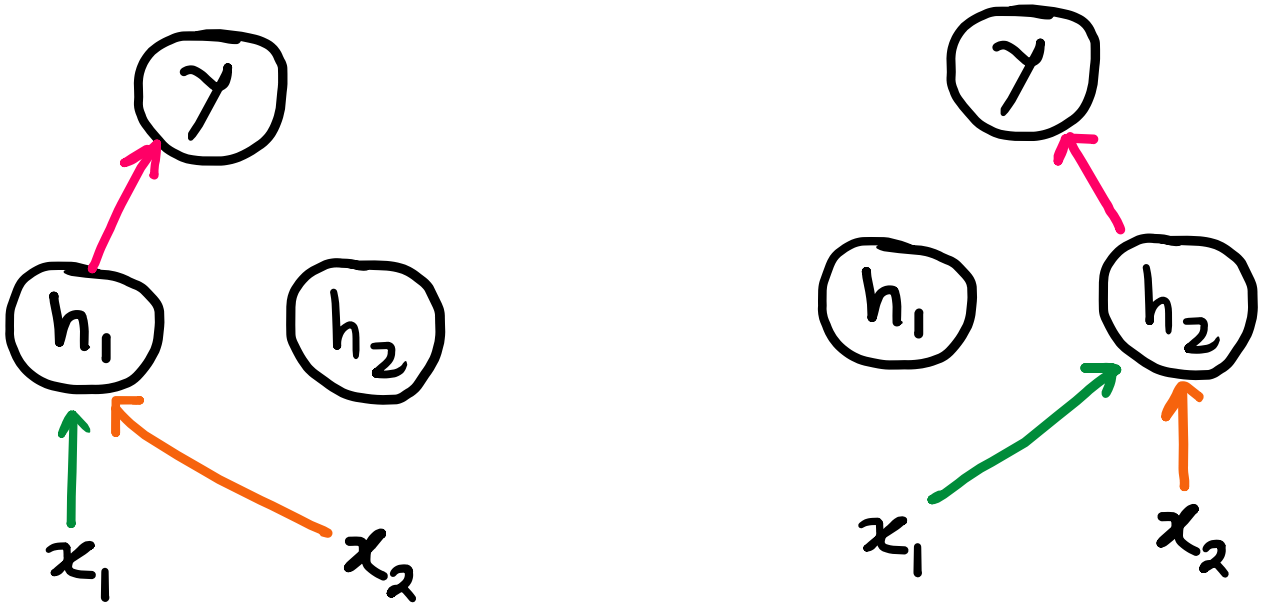
- Convex functions are very convenient for optimization since starting from any point in the parameter space, gradient descent will eventually reach the minimum

- Non-convex functions have multiple local minima



Local optima

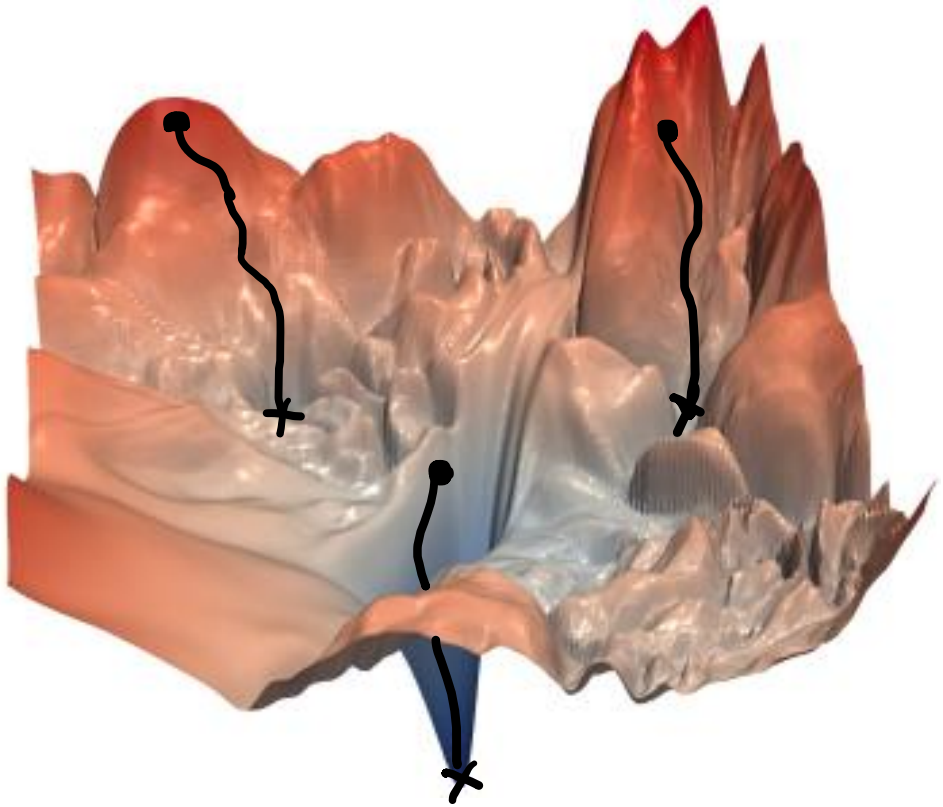
- Unfortunately, training a neural network with hidden units is **non-convex**, mainly because of *permutation symmetries*



- **Permutation symmetry**: We can re-order the weights in a way that gives the same loss function for the neural network

Local optima

- Training a multilayer neural network with hidden units will have multiple local minima



- Gradient descent starting from two different initializations may converge to different local optima

Local optima

- In general, it is very hard to diagnose if you are stuck in a bad local minimum
- **Workaround**: One can try to improve the issue by using **random restarts**
- **Random restarts:** initialize the training from several random locations, run the training procedure from each one, and pick whichever result has the lowest cost
- Random restart is sometimes done in neural net training, but more often we just ignore the problem
- In practice, the local optima are usually fine, so we think about training in terms of converging faster to a local minimum, rather than finding the global minimum

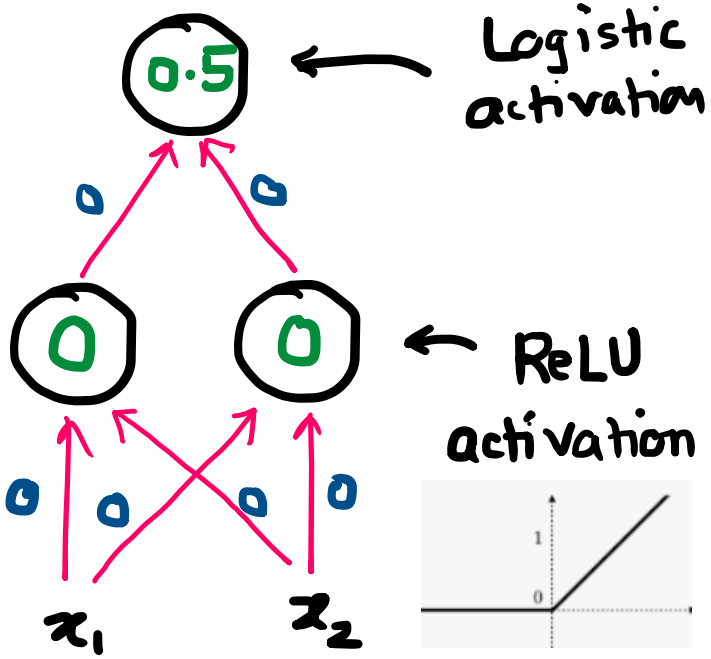
Gradient descent-based optimizers

Problems and workarounds

Symmetry

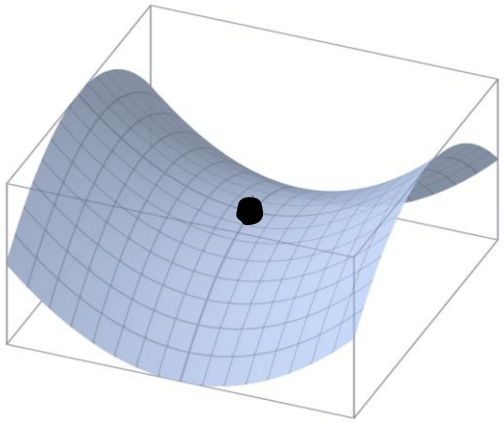
Symmetry

- We start our optimization by initializing the values of weights and biases
- Suppose we initialize all the weights and biases of a neural network to all zeros
- All the hidden activations will be identical (indistiguisable features), and all the weights feeding into a given hidden unit will have zero derivatives
- No learning will occur
- If the initial weights are zero, multiplying them by any gradient will set the gradients to be zero. Due to zero gradient, there will be no change in the weights

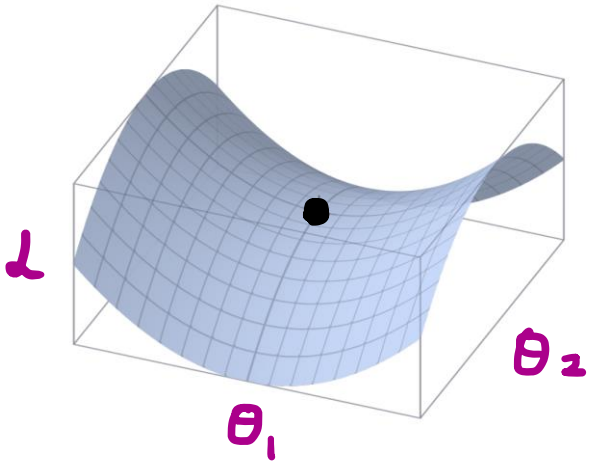


Symmetry problem

→ Saddle points



Saddle point



- A saddle point has $\frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}} = \mathbf{0}$, even though we are not at a minimum
- We are at a location which is minimum with respect to some directions, and maximum with respect to others
- When would saddle points be a problem?
  - If we're exactly on the saddle point, then we're stuck
  - If we're slightly to the side, then we can get unstuck

Initialization strategies

- **Don't initialize all the weights and biases to zero**!

Idea 1: Constant initialization

`torch.nn.init.`

- *Result*: For fully connected neural network: identical gradients, identical neurons. Bad! Because we want to learn different features

Idea 2: Random weights from a standard Gaussian distribution to break symmetry

Popular initialization schemes

- **Xavier** Init: For **Sigmoid/Tanh** activation

$$W^{[l]} \sim \mathcal{N}(0, \frac{2}{n^{[l-1]}+n^{[l]}})$$

$$b^{[l]} = 0$$

variance

# of neurons in layer $l-1$

- **Kaiming He** Init: **ReLU** activation

$$W^{[l]} \sim \mathcal{N}(\mu = 0, \sigma^2 = \frac{2}{n^{[l-1]}})$$

$$b^{[l]} = 0$$

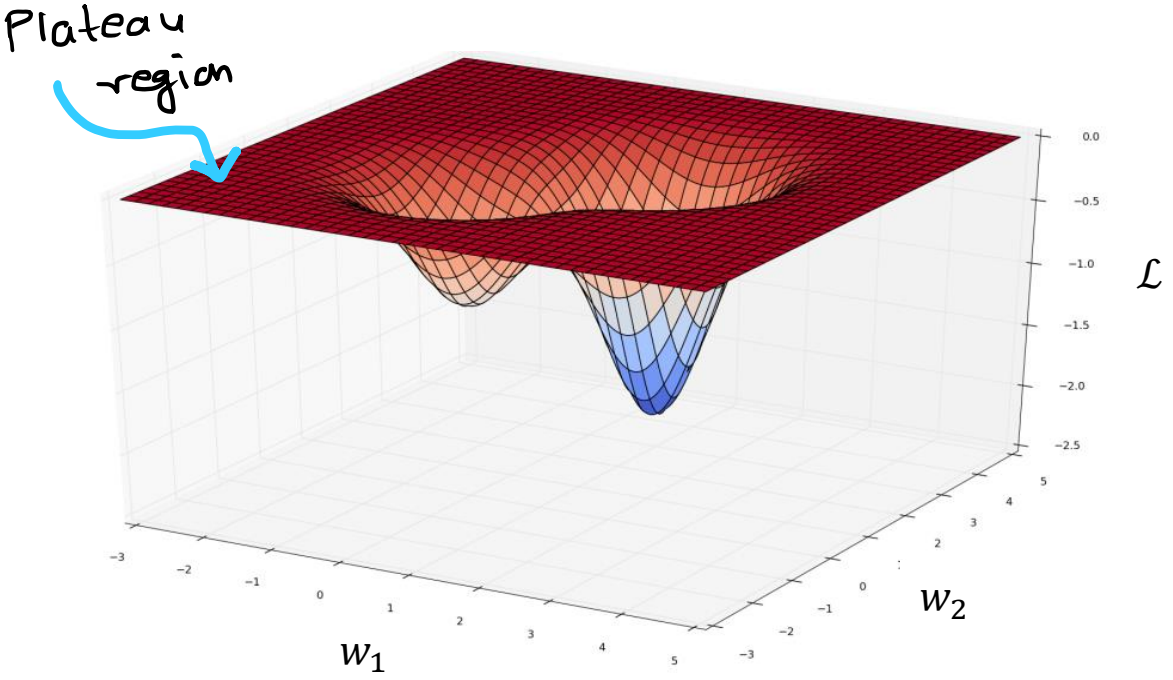
Gradient descent-based optimizers

Problems and workarounds

Saturated and dead units

Saturated and dead units

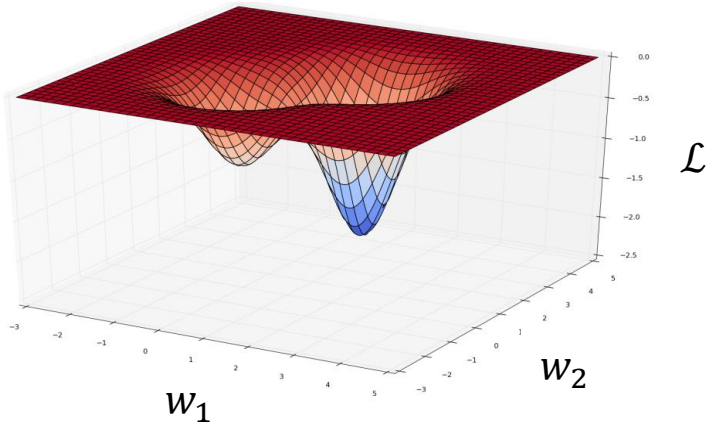
- A flat region of the loss surface is called a **plateau**



- **Caused by**
  - **Saturated units** whose activations are always near the ends of their dynamic range/possible values
  - **Dead units** whose activations are always close to zero

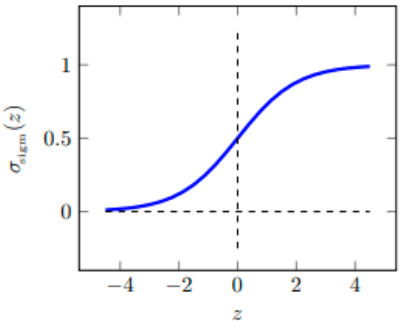
Saturated and dead units

- A flat region of the loss surface is called a **plateau**

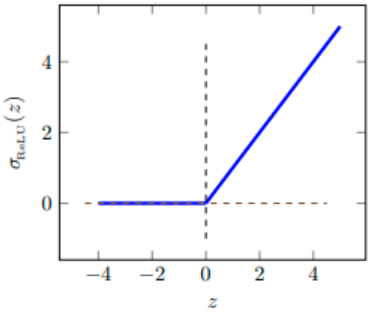


Examples

- Logistic activation



- ReLU activation

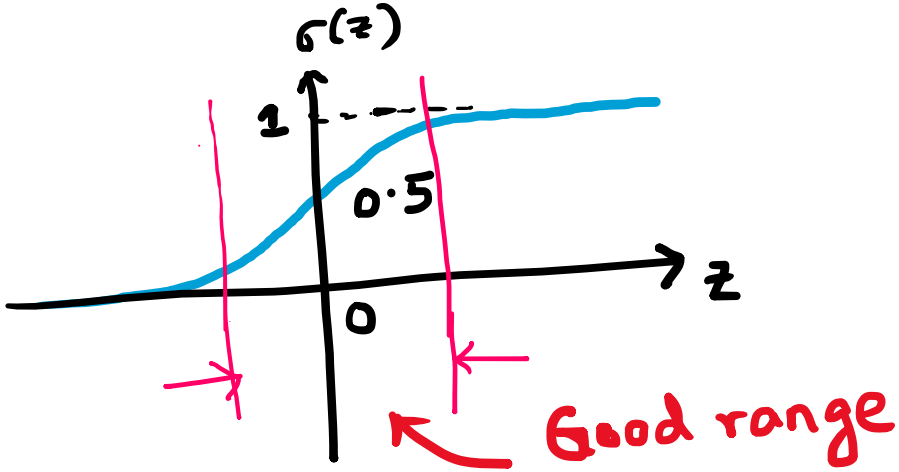


- **Caused by**
  - **Saturated units** whose activations are always near the ends of their dynamic range/possible values
  - **Dead units** whose activations are always zero (or close to zero)

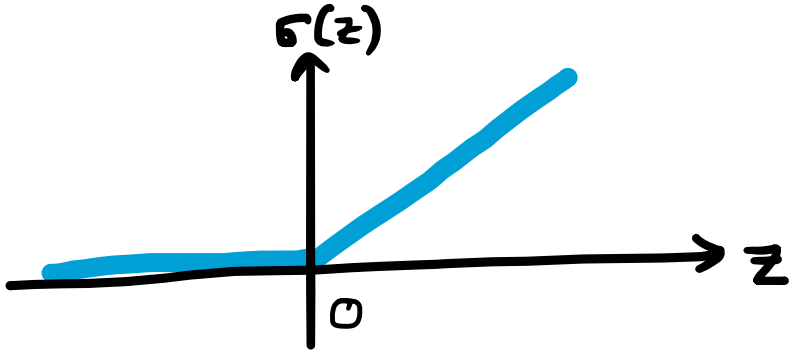
- **Problem:** Gradient signal is zero, cannot update the weights

Saturated and dead units: Workaround

- Choose the scale of the random initialization of the weights so that the pre-activations are in the middle of their domain, e.g. Xavier initialization for sigmoid activations



- ReLU units don't saturate for positive z, which is convenient. Unfortunately, they can die if z is consistently negative, so it helps to initialize the biases to a small positive value (such as 0.1)



$$z = W^T x + b$$

+ve value

Gradient descent-based optimizers

Problems and workarounds

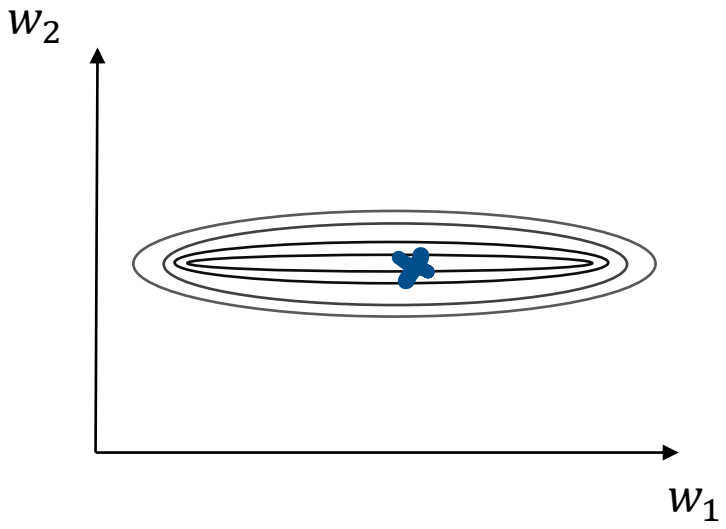
Ill-conditioning

Ill-conditioning

- Suppose that we have the following dataset for linear regression

$$y = w_1 x_1 + w_2 x_2 + b$$

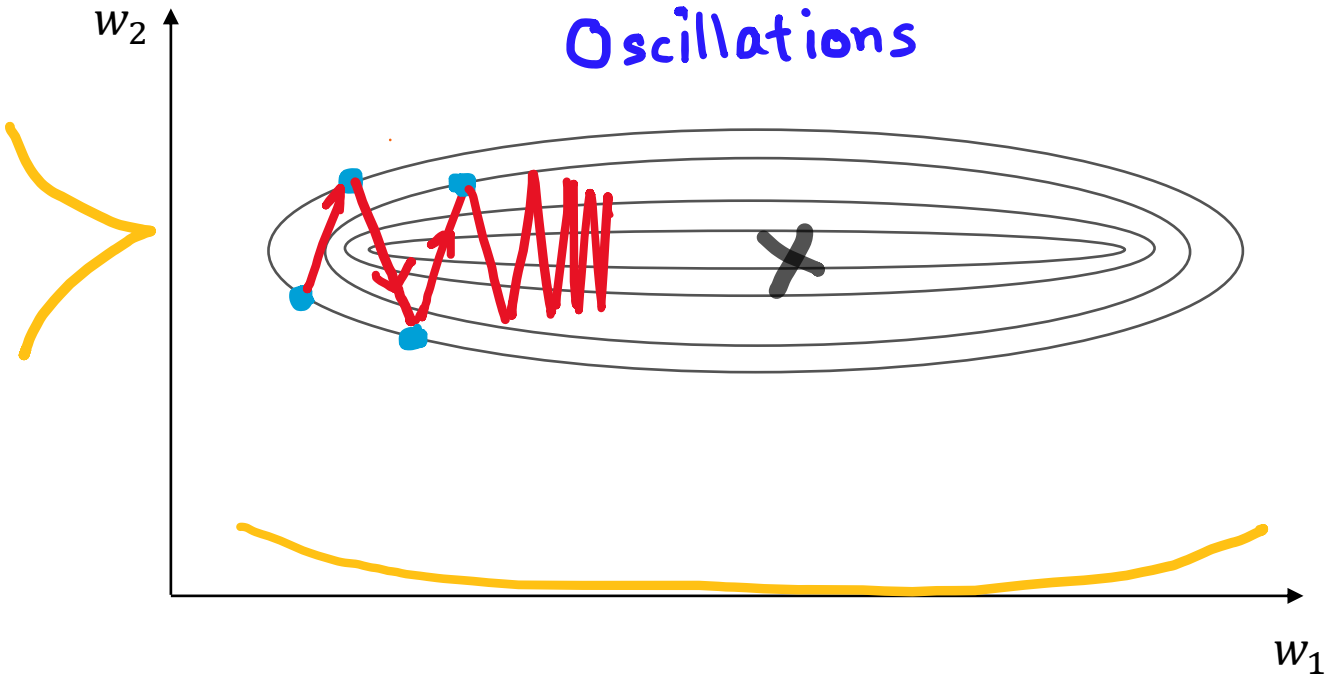
x_1	x_2	t
0.0023	1002.3	5.12
0.0025	1005.2	4.25
0.0016	988.7	3.45
0.0013	945.9	3.13
⋮	⋮	⋮



- What kind of values will w_1 and w_2 take?
- w_1 will take larger values, whereas w_2 will take very small values

Ill-conditioning

- Which weight, w_1 or w_2, will receive a larger gradient descent update?
  - w_2 receives a much larger update due to steeper gradient
  - w_1 receives a much smaller update due to smaller gradient



- But, here we want to take smaller steps in the steeper direction and larger steps in the flatter direction

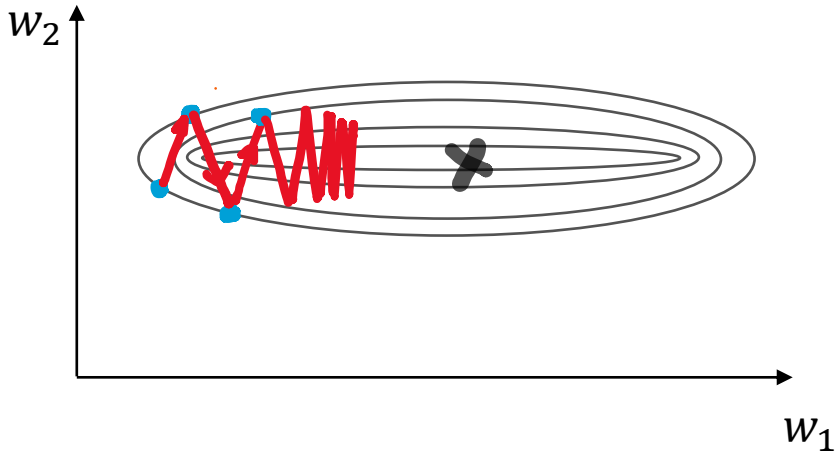
Ill-conditioning: Workarounds

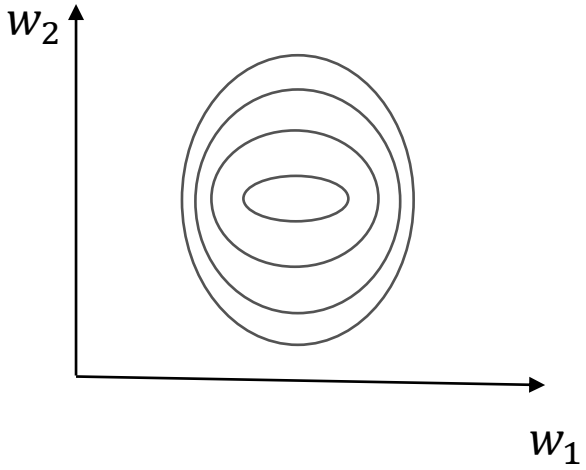
- We want larger gradient descent updates in the direction of w_1 and smaller in the direction of w_2

- **One workaround: Normalization**

- Center the inputs to zero mean and unit variance

$$\tilde{x}_j = \frac{x_j - \mu_j}{\sigma_j}$$

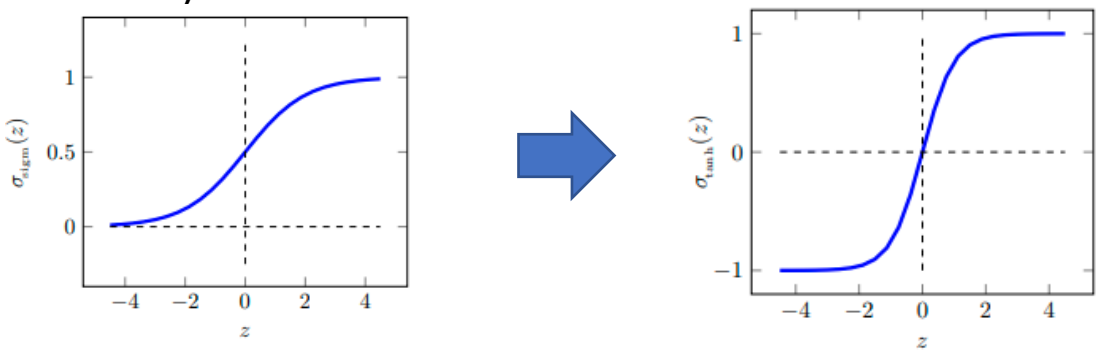




Ill-conditioning: Workarounds

- Not just inputs, but inputs to several hidden layers need to be centered too

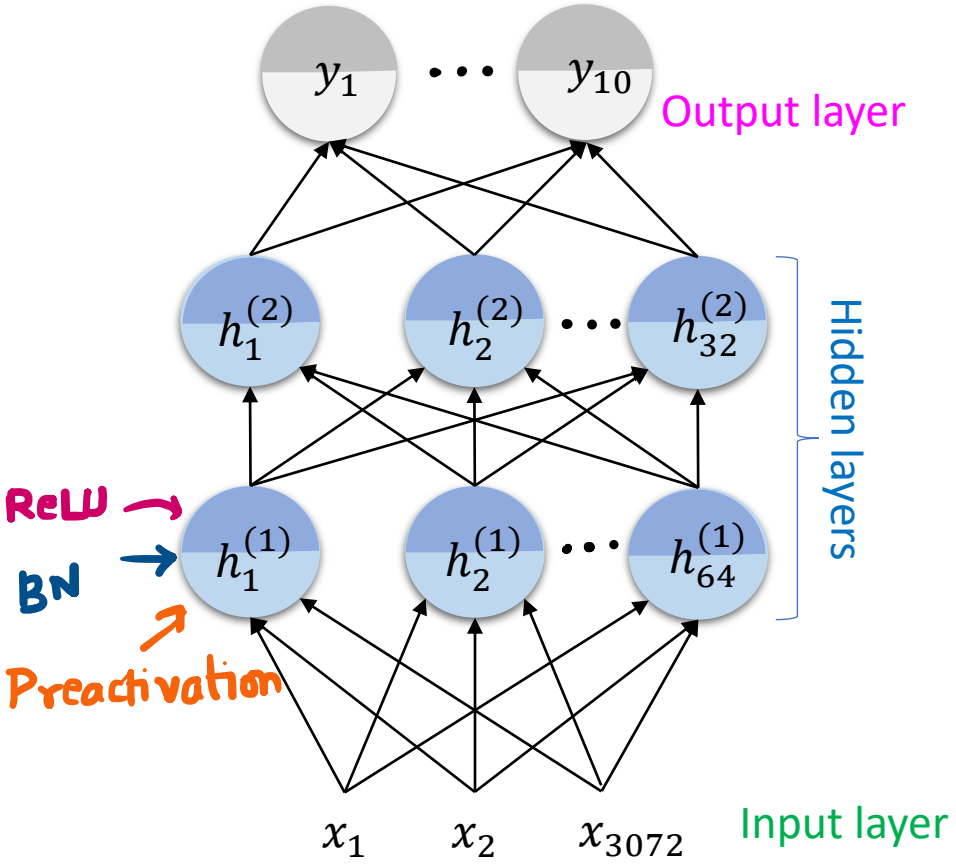
- Hidden units may also have non-centered activations, and those are even harder to deal with
  - One trick: replace logistic units (which range from 0 to 1) with $\tanh$ units (which range from -1 to 1)



  - Another trick: **Batch normalization**
    - It centers each hidden activation and can speed up training by 1.5-2x
    - It normalizes the activations of each layer to unit-variance Gaussians
    - Is applied immediately *after* fully connected/conv layers and *before* non-linear activations

Batch normalization example

```
class MLP(nn.Module):
  '''
    Multilayer Perceptron.
  '''
  def __init__(self):
    super().__init__()
    self.layers = nn.Sequential(
      nn.Flatten(),
      nn.Linear(32 * 32 * 3, 64),
      nn.BatchNorm1d(64),
      nn.ReLU(),
      nn.Linear(64, 32),
      nn.BatchNorm1d(32),
      nn.ReLU(),
      nn.Linear(32, 10)
    )
```



https://www.machinecurve.com/index.php/2021/03/29/batch-normalization-with-pytorch/

Recap of what we have seen

Problem	Diagnostics	Workarounds
Local optima	Hard to diagnose	Random restarts
Symmetries	Check $\mathbf{W}$, $\boldsymbol{b}$	Initialize properly
Dead/Saturated units	Plot activation histograms	Initial scale of **W;** ReLU
Ill-conditioning	(hard)	Batch normalization, momentum, ADAM